

Strata — BA Guide

For business analysts

strata-kb v1.3.0 · 2026-09-19

1. Who this guide is for

You write requirements. Tickets, user stories, acceptance criteria — and for large features, mission plans that break them down.

Your problem is not writing. It is **grounding**: making sure that when a ticket says "the field is 4 characters, alphanumeric", that claim comes from a real document, at a known revision, and stays verifiable months later when someone asks where it came from.

Strata gives you three things for that:

1. **Search over published knowledge**, so you can find the passage instead of remembering it.
2. **Citations that pin a version**, so a ticket records exactly what the source said the day you wrote it.
3. **A Definition-of-Ready gate**, so a ticket cannot reach a developer with a broken or missing citation.

1.1 What a BA repo does and does not do

Does	Does not
Read the hub's published knowledge	Ingest documents
Cite it, pinned to a hub commit	Summarize anything
Version tickets and missions in Git	Publish knowledge
Gate them in CI	Push to your issue tracker

Your repository is strictly **read-only** against the knowledge base. You cite, you never contribute. That is deliberate: it keeps requirements traceable to a knowledge version without giving the requirements process a way to change the knowledge itself.

The assistant also never opens a ticket for you. It produces Markdown; you read it, commit it, and paste it into your tracker yourself.

2. Setup, once

```
pip install strata-kb
kb init --kind ba
```

2.1 Point at the hub

```
# .kb/config.yaml
kind: ba
hub: https://github.com/acme/kb-hub.git
```

This is what `kb ticket lint` and `kb query` resolve references against.

2.2 Connect your assistant to the hub

`.mcp.json` (Claude Code) and `.cursor/mcp.json` (Cursor) were scaffolded already wired to the hub's search and citation tools, but through two placeholders — `${STRATA_KB_HUB_URL}` and `${STRATA_KB_HTTP_TOKEN}` — that nothing has set yet.

Run `kb mcp-setup` (in your assistant: `/kb-mcp-setup`). It asks for the hub's HTTP base URL and token — ask your hub maintainer for the token — writes both into `.env`, makes sure git ignores that file, and then verifies them against the hub, so a wrong URL and a rejected token come back as different errors:

Variable	Example
<code>STRATA_KB_HUB_URL</code>	<code>http://kb-hub.example.com:8321</code>
<code>STRATA_KB_HTTP_TOKEN</code>	the hub's bearer token

A bare re-run of `kb mcp-setup` reads both values back out of `.env` and verifies again, so checking your connection later never means retyping the token. Prefer the hidden prompt or the `STRATA_KB_HTTP_TOKEN` environment variable over the command's `--token` flag — that flag leaves the token sitting in your shell history.

`.mcp.json` and `.cursor/mcp.json` read those two variables from your environment, not from `.env` directly, so load `.env` into your shell (`set -a; source .env; set +a`, or use `direnv`) and restart your assistant — MCP reads the environment only at startup.

2.3 Open the repository in your assistant

`ba-ticket-author` and `ba-mission-plan` are scaffolded as skills, commands and prompts for **Claude Code**, **GitHub Copilot** and **Cursor**. Whichever you use, the pipeline is the same.

2.4 Configure the CI gate

Setting	Where	What it is
<code>STRATA_KB_HUB</code>	Actions → Variables	The hub the gate resolves refs against
<code>KB_HUB_TOKEN</code>	Actions → Secrets	A read token — only needed for a private hub
<code>KB_FAIL_ON_STALE</code>	Variables , optional	Set to any value to make an upstream amendment fail the gate

Then, in branch protection, **require the `kb-ticket-lint` check** on the branch your tickets land on. Lint running in CI does not block a merge by itself; requiring the check is what does.

A pull request opened **from a fork** cannot read repository secrets, so on a private hub the gate fails there with a hub-unreachable message. That is the gate refusing to go green without checking, not a network fault. Merge fork contributions through a branch in this repository instead.

3. Writing a ticket

Invoke `/ba-ticket-author` and describe the business need. The agent runs a nine-step pipeline; your job is steps 1, 3 and 9.

1 Intake	you describe the need
2 Parent mission	optional – name the mission this story belongs to
3 Ground	<code>kb_search</code> ; YOU pick which sections apply
4 Draft	story, ACs, use cases, two diagrams
5 Pin	<code>kb_context_new</code> embeds the pinned block
6 Lint	<code>kb ticket lint</code> until DoR: PASS
7 Ground technical	the SA fills Technical grounding; <code>kb ticket check</code>
8 Maturity review	two independent reviews, up to 3 rounds
9 Review → save	you read it, commit it, paste it into the tracker

3.1 Intake

Describe the need in business terms: the capability, who needs it, and the value. Resist writing the solution. The agent's later steps work better from a clear problem than from a pre-decided implementation.

3.2 Grounding — the step that actually matters

The agent calls `kb_search` and shows you **every candidate section**, not just its favourite. Read them. Pick the ones that genuinely apply.

This is the only step where your domain judgement cannot be delegated. The agent can find candidate passages; it cannot know that §4.12 is superseded in practice by a local policy, or that two similar sections differ in a way that matters here. Everything downstream — the draft, the pin, the developer's implementation — rests on this choice.

3.3 Drafting

The agent fills the ticket template: summary, user story, background, acceptance criteria, use cases, and two Mermaid diagrams (a sequence diagram and a business flow). Every claim that touches a standard carries a `[doc-id $section]` citation.

3.4 Pinning

Once you confirm the sections, the agent calls `kb_context_new` and embeds the returned `## KB context` block. That block records the hub's commit at this moment. From now on the ticket does not say "see the handbook" — it says "this is exactly what the handbook said on the day this was written, and here is how to check."

3.5 Lint

`kb ticket lint` runs, the agent fixes what it reports, and it repeats until `DoR: PASS`.

3.6 Maturity review

Once lint passes, the agent runs two independent reviews against `docs/review-rubric.md` — one scoring **Business coverage**, one scoring **Dev implementability**. It applies fixes and reviews again, up to three rounds or until both axes reach 4 or higher.

A gap it cannot close itself becomes an owned open question, written `OPEN(<owner>)`, rather than a guess. The outcome lands in the ticket's `## Review record` section.

Read the open questions. They are the agent telling you where it knew it did not know.

3.7 Your review

The draft lands in `tickets/<ticket-id>.md`. You read it, commit it, and paste it into the tracker. The assistant never does that for you.

3.8 SA grounds the technical half

Step 7 runs this for you. Once lint reports `DoR: PASS`, the agent saves the draft and invokes `/sa-ticket-ground` on it as its own subagent. It fills the SA-owned `## Technical grounding` section — service, files, tables, routes, externals, volumes, healthcheck commands and device reservations, test command — each a section id from the hub's `<repo>-code` document. It never edits your sections; a business statement that contradicts the code facts is quoted there, not corrected.

Code that does not exist yet is not missing data, it is a design decision. The SA proposes a row in the parent mission's `## Technology decisions` (status `OPEN`, a human owner) and writes `[NEW: D<n>]` pointing at it. Only code that exists and the document cannot prove — internal flow, failure modes, request bodies — goes under `Open decisions` for the developer.

`kb ticket check tickets/<ticket-id>.md` gates it: `PASS` only when every id resolves, every `[NEW: D<n>]` names a `DECIDED` row, and `Open decisions` is empty. A ticket with no parent mission has no table to hold a row, so the SA writes free-text `[NEW: <reason>]` instead — the gate accepts it as a note; a mission would give the decision an owner. An `OPEN` row fails and names its owner — flipping it to `DECIDED` is your call, never the agent's. The handover carries a `## Needs input` block listing exactly those rows.

Run `/sa-ticket-ground` by hand only to re-ground a ticket after `<repo>-code` has moved.

4. Mission plans — for large features

A feature spanning several user stories gets a **mission plan** first. Small work goes straight to a ticket. A mission is never mandatory.

```
/ba-mission-plan
```

The pipeline is **Intake** → **Ground** → **Draft** → **Split** → **Ground services** → **Pin** → **Lint** → **Maturity review** → **Review**, and it saves `missions/M-<slug>.md`.

A mission carries:

- A **C4 Level 1** diagram (System Context) and a **C4 Level 2** diagram (Container).

- A **scope split** — what is in, what is explicitly out.
- A **user-story backlog** whose ids derive from the mission id.

```
kb mission lint missions/M-checkout-revamp.md
```

The gate enforces: required structure, both diagrams present, a well-formed backlog whose ids derive from the mission id, and every citation resolving at the pinned hub version. A `0/N US drafted` coverage warning is normal — the tickets do not exist yet.

What stays **your** judgement: whether the backlog is actually complete.

4.1 Linking tickets to a mission

When drafting a story from the backlog, name the parent mission. The agent then:

- reads `missions/<mission-id>.md` and takes the story title from the backlog row,
- writes `> Parent mission: <mission-id>` on its own line directly under the ticket title,
- saves the ticket as `tickets/<mission-id>-US<n>.md`.

That filename is what the back-link check looks for. Dropping the parent line is valid — the ticket just becomes untraceable to its mission.

4.2 Numbering

Backlog numbering gaps are fine. If you drop a story, leave its number retired. Renumbering would break the filenames of tickets already drafted.

4.3 SA grounds the service list

Step 5 runs this for you. Once you confirm the backlog and `## Sequencing` is filled, the agent invokes `/sa-ticket-ground --mission` and it fills `## Services & order` — one row per `svc.<name>` from the hub's `<repo>-code` document, with `Depends on` copied from that record. A service the mission will create carries `[NEW: D<n>]`, pointing at a row the SA appends to `## Technology decisions` for you to decide. The gate then reports FAIL on exactly those rows until you flip them to `DECIDED` — that is the expected result, not a defect. Capability layer only: no file names, no tables, no routes — those belong to each ticket's own `## Technical grounding`, filled later at §3.8.

5. Using code knowledge

Besides domain documents, the hub holds two documents per product repository, published by that repository's own workflow: `<repo>-code` (generated structure — `svc.*`, `db.*`, `api.*`, `int.*`, `cmd.*`, `struct.tree`) and `<repo>-svc` (curated responsibility — what each service is actually for).

The `ba-ticket-author` and `ba-mission-plan` skills do not read either document themselves. Where a ticket or mission needs a service, table, route or file name, they write `%%TODO: verify against codebase%%` with an owned open question instead of guessing, and `/sa-ticket-ground` (§3.8, §4.3) answers those from `<repo>-code` in the SA-owned section — `## Technical grounding` in a ticket, `## Services & order` in a mission. Every id it writes is checked against the document by `kb ticket check`; what the document cannot prove — internal flow, failure modes, request bodies — is parked under `Open decisions` for the developer, who has the code.

The section reference itself, spelled the way a real citation is:

```
<repo>-code $svc.<name>
<repo>-svc $svc.<name>
```

One caution. `<repo>-svc` grounds a diagram. It is never a substitute for a domain citation in an acceptance criterion. A code, format, enum or threshold that encodes a standard must still come from a pinned domain section, not from a service's responsibility text.

6. What the gate checks — and what it does not

6.1 Errors (the gate fails)

- **Required sections present:** Summary, User Story, Background, Acceptance Criteria, Use cases, both Mermaid diagrams, KB context, Definition of Ready.
- **At most 10 acceptance criteria.** More than ten `- [ ]` items under `## Acceptance Criteria` fails the gate. Split the ticket — merging two conditions into one AC to fit under the cap is the thing the rule exists to stop.
- **Exactly one user story.** More than one `As a ... I want ... so that ...` shape under `## User Story` fails the gate. One ticket per story.
- **Every `## KB context` ref resolves** at its pinned hub commit. No broken or malformed refs.
- **Every inline `[doc-id $section]` citation is backed by a pinned ref.**

A citation written in the old bare form still counts, with a warning telling you to bracket it.

6.2 Warnings (still your judgement)

Warning	Why it is not an error
An acceptance criterion touching a standard has no citation	The tool cannot reliably tell which criteria touch a standard
The business quality of the story	Not machine-checkable
<code>## Review record</code> missing, empty, or still a placeholder	Lint can see the section, not whether the review was meaningful
A stale ref	The cited section was amended upstream. Still resolves, so not broken — but you should look.
A pinned ref the body never cites	The pin may be background the ticket did not need to quote
An acceptance criterion prescribes a shell command (<code>docker compose ...</code> , <code>curl ...</code> , <code>grep ...</code> , or two commands chained with <code>&&</code>)	The tool flags the shape; whether the AC should instead state the observable outcome is your call — the command itself belongs in <code>## Test data & verification</code> or the plan

6.3 Making staleness fail

```
kb ticket lint tickets/ABC-123.md --fail-on-stale
kb mission lint missions/M-checkout.md --fail-on-stale
kb resolve tickets/ABC-123.md --status-only
```

Or set the `KB_FAIL_ON_STALE` repository variable to make the CI gate do it for every ticket.

6.4 Why the workflow is not path-filtered

The scaffolded workflow is named `kb-ticket-lint` and triggers on every pull request, not only on changes under `tickets/` or `missions/`. That is deliberate.

GitHub never synthesises a passing status for a job that never started. A `paths:` filter on a *required* check would leave any pull request touching neither directory waiting forever for a check that will never run. Instead the job always starts, inspects the diff, and dispatches by directory — exiting cleanly with a notice when there is nothing to lint.

The workflow name also predates the mission gate. It is kept for branch-protection compatibility; renaming it would leave already-protected repositories blocked on a check that no longer exists.

7. When a citation goes stale

Knowledge changes. A ticket written in March against a document amended in May must not silently start meaning something else.

```
kb resolve tickets/ABC-123.md      # ok | stale | broken, per ref
kb diff hr-handbook --against <rev> # what actually changed
```

Status	Meaning	What to do
<code>ok</code>	The pinned content still matches what is published	Nothing
<code>stale</code>	The source was amended after you cited it	Run <code>kb diff</code> ; decide whether the acceptance criteria need updating
<code>broken</code>	The reference no longer resolves at all	Re-ground and re-pin with <code>kb context new</code> or <code>kb_context_new</code>

`kb diff` tells you exactly what moved: section titles, one-line summaries, the summary text, the full original, sections added and removed, and whether the document's ordering changed.

Most staleness is harmless — a typo fix upstream. Some of it is a requirement that no longer matches reality. The tool tells you which sections to look at; the judgement is yours.

8. Working without an assistant

Everything the agent does, you can do at a terminal.

```
kb query "refund eligibility window" --tags payments
kb get payments-spec 4.2 --level 12
kb context new --refs "payments-spec $4.2, payments-spec $4.3"
kb ticket lint tickets/ABC-123.md
kb mission lint missions/M-checkout.md
```

`kb context new` prints the block; paste it into the ticket under `## KB context`.

9. Troubleshooting

Symptom	Cause	Fix
Lint: "ref does not resolve"	The section id is wrong, or that content is not published	Re-run <code>kb query</code> and re-pin. Check the hub PR actually merged.
Lint: "citation not backed by a pinned ref"	The body cites something the KB context block does not pin	Add the ref via <code>kb context new</code> , or remove the citation
Lint: "missing required heading"	A required section is absent — or present only inside a fenced code block	Move the real heading outside the fence
Lint passes locally, fails in CI	<code>STRATA_KB_HUB</code> is unset in Actions variables, or the hub is private and <code>KB_HUB_TOKEN</code> is missing	Configure both; see §2.4
The gate fails on a fork PR	Forks cannot read repository secrets	Merge through a branch in this repository
The assistant cannot reach the hub	<code>STRATA_KB_HUB_URL</code> or <code>STRATA_KB_HTTP_TOKEN</code> unset or wrong	Re-run <code>kb mcp-setup (/kb-mcp-setup)</code> — it diagnoses which one, and a bare re-run re-verifies without retyping the token
<code>kb mcp-setup</code> fails with "token was rejected" after the hub maintainer gave you a fresh one	A bare re-run reads the <i>old</i> token straight back out of <code>.env</code> — the prompt only appears when nothing is on disk yet	<code>STRATA_KB_HTTP_TOKEN=<new-token></code> <code>kb mcp-setup</code> , or delete the <code>STRATA_KB_HTTP_TOKEN</code> line from <code>.env</code> and re-run
<code>kb query</code> finds nothing	The document is not published yet, or the tags are too narrow	Drop <code>--tags</code> ; check with the hub owner that the publish PR merged
Everything resolves <code>broken</code> after an upgrade	The ticket carries a pin from an older block format	Re-pin with <code>kb context new</code>

10. Command summary

Command	Purpose	Exit
<code>kb mcp-setup [--hub-url URL] [--no-verify]</code>	Write the hub's HTTP MCP credentials into <code>.env</code> and verify them	<code>0</code> ok, <code>1</code> no value/probe failed
<code>kb query <text> [--tags t]</code>	Search published knowledge	<code>0</code>
<code>kb get <doc> <section></code>	Fetch one section	<code>0</code>
<code>kb tags</code>	List the published tag vocabulary	<code>0</code>
<code>kb context new --refs "..."</code>	Generate a pinned citation block	<code>0</code> ok, <code>1</code> unresolvable ref
<code>kb resolve <file></code>	Check a ticket's refs	<code>0</code> all ok, <code>1</code> broken, <code>2</code> stale
<code>kb diff <doc> --against <rev></code>	What an amendment changed	<code>0</code>
<code>kb ticket lint <file></code>	Ticket Definition-of-Ready gate	<code>0</code> PASS, <code>1</code> FAIL, <code>2</code> stale
<code>kb mission lint <file></code>	Mission Definition-of-Ready gate	<code>0</code> PASS, <code>1</code> FAIL, <code>2</code> stale

Slash commands: `/ba-ticket-author`, `/ba-mission-plan`.

11. Upgrading the repository

```
kb init --kind ba
```

This touches **scaffold files only** — the CI workflow, the skill, command and prompt wrappers, and the templates under `docs/` — and overwrites one only when its content differs from the new template.

`.kb/config.yaml`, `.kb/index.yaml` and `.claude/settings.json` are never touched by a plain `kb init`.

--force replaces all three outright, unmerged: your hand-edited `.kb/config.yaml` including `kind`, `repo_id` and `asset_store`; `.kb/index.yaml`; and your entire `.claude/settings.json` including hooks, permissions and model. Use it deliberately.

Your own content under `tickets/` and `missions/` is not scaffolding. `kb init` never reads, writes or overwrites anything you author there — the only thing it places in those directories is an empty `.gitkeep`, so Git can track them before your first ticket exists.